1. cloning the repository

```
PS C:\Users\Hp> git clone https://github.com/PreethiJC/Information-Retrieval
Cloning into 'Information-Retrieval'...
remote: Enumerating objects: 226, done.
Receiving objects:  15% (34/226), 76.01 KiB | 15.00 KiB/s
```

2.

```
[2026-04-27T23:07:28,949][WARN ][o.e.h.AbstractHttpServerTransport] [SAMUEL] caught exception while handling client http
 traffic, closing connection Netty4HttpChannel{localAddress=/[0:0:0:0:0:0:0:1]:9200, remoteAddress=/[0:0:0:0:0:0:0:1]:23
78}io.netty.handler.codec.DecoderException: javax.net.ssl.SSLHandshakeException: (certificate_unknown) Received fatal al
ert: certificate_unknown
        at io.netty.codec@4.1.130.Final/io.netty.handler.codec.ByteToMessageDecoder.callDecode(ByteToMessageDecoder.java
:515)
        at io.netty.codec@4.1.130.Final/io.netty.handler.codec.ByteToMessageDecoder.channelRead(ByteToMessageDecoder.jav
a:296)
        at io.netty.transport@4.1.130.Final/io.netty.channel.AbstractChannelHandlerContext.invokeChannelRead(AbstractCha
nnelHandlerContext.java:444)
        at io.netty.transport@4.1.130.Final/io.netty.channel.AbstractChannelHandlerContext.invokeChannelRead(AbstractCha
nnelHandlerContext.java:420)
        at io.netty.transport@4.1.130.Final/io.netty.channel.AbstractChannelHandlerContext.fireChannelRead(AbstractChann
elHandlerContext.java:412)

See logs for more details.
```

- Elasticsearch was started using elasticsearch.bat.
- The server successfully initialized security features and reported cluster health as GREEN, indicating that the system was fully operational.
- Accessing http://localhost:9200 in a web browser returned a JSON response confirming that the Elasticsearch service was running. Here is the confirmation:-

Not secure ~~https~~://localhost:9200

retty-print


```
"name" : "SAMUEL",
"cluster_name" : "elasticsearch",
"cluster_uuid" : "Ih0kgALIRUaKpAk5DD7q8A",
"version" : {
  "number" : "9.3.3",
  "build_flavor" : "default",
  "build_type" : "zip",
  "build_hash" : "640408e2dfd2af9fbfe5079e1575f93d8909a5f5",
  "build_date" : "2026-04-01T22:08:18.783399214Z",
  "build_snapshot" : false,
  "lucene_version" : "10.3.2",
  "minimum_wire_compatibility_version" : "8.19.0",
  "minimum_index_compatibility_version" : "8.0.0"
},
"tagline" : "You Know, for Search"
```

4. Installing kibana :

Download the appropriate package from elastic.co/downloads/kibana, extract/install it, configure kibana.yml to connect to Elasticsearch, and start the service.

5.Installing and checking python version

```
PS C:\Users\Hp> python --version
Python 3.12.7
```

- Elasticsearch installation

```
Installing collected packages: elastic-transport, elasticsearch
Successfully installed elastic-transport-9.2.1 elasticsearch-9.3.0

[notice] A new release of pip available: 22.2.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

- Nltk installation

```
Successfully installed nltk-3.9.4

[notice] A new release of pip available: 22.2.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

- Numpy installation

This project builds an information retrieval pipeline for the CACM collection, generates retrieval runs with multiple scoring models, and evaluates them with standard ranking metrics. The workspace is located at C:\Users\Hp\Downloads\HW6.

Objectives

- Parse the CACM document collection, query file, and qrels.
- Produce per-model TREC-formatted run files and a static feature matrix CSV.
- Evaluate the run files with real metric values for inclusion in the report.

Problem Statement

The original codebase was written around an AP-style workflow. The task was to switch the project to the CACM dataset, make the pipeline work with CACM's line-based file format, and produce a report with real retrieval values.dcs.

System Requirements

- OS: Windows.
- Python 3.10+ inside the project virtual environment (.venv).
- Python packages from requirements.txt, including elasticsearch, elasticsearch-dsl, pandas, and scikit-learn.
- No Perl dependency was required for the final values because evaluation was computed directly in Python when trec_eval.pl was unavailable.

Tools & Technologies Used

- Python scripts: ir_pipeline.py, build_feature_matrix.py, Feature_Matrix.py (legacy), train_ranker.py.
- Libraries: scikit-learn, pandas, optional elasticsearch, optional dill.
- Dataset: CACM files in cacm_dataSet/.
- Evaluation: TREC-style metric calculation for MAP, nDCG@10, and P@10.

Methodology

1. Parse the CACM documents and queries from the line-based .I/.T/.W format.dcs.
2. Build an in-memory index over the CACM document collection.
3. Score each query with TF-IDF, Okapi TF, and BM25.
4. Write the feature matrix CSV and TREC run files.
5. Compute MAP, nDCG@10, and P@10 for each run and save the results.

```
(.venv) PS C:\Users\Hp\Downloads\HW6> Get-Content .\cacm_evaluation_summary.txt -Raw
TF-IDF_Results_File.txt: MAP=0.170144, nDCG@10=0.268293, P@10=0.180769
OkapiBM25_Results_File.txt: MAP=0.286610, nDCG@10=0.427835, P@10=0.288462
OkapiTF_Results_File.txt: MAP=0.159165, nDCG@10=0.285230, P@10=0.209615
```

Project Structure

- build_feature_matrix.py — CLI entry point for generating the feature matrix and TREC outputs.
- ir_pipeline.py — parsing, indexing, scoring, and run writing.
- Feature_Matrix.py — legacy script, patched to be more tolerant of missing dependencies and hardcoded paths.
- train_ranker.py — learning-to-rank script based on the generated CSV.
- cacm_dataSet/ — CACM collection files: cacm.all, query.text, qrels.text, cite.info, common_words.
- cacm_staticFeatureMatrix.csv — generated CACM feature matrix.
- cacm_outputs/ — generated CACM run files.

Implementation Details

- Parsing: ir_pipeline.py now detects CACM's line-based format and parses. I, .T, .A, .B, .N, and .W sections.github+1
- Queries: CACM query IDs are padded to match the qrels format.
- Qrels: CACM qrels are treated as listed relevant document pairs.
- Scoring: InMemoryIndex uses an inverted index for TF-IDF, Okapi TF, and BM25, which made the CACM build practical.acm+1
- Output: build_feature_matrix.py writes the feature matrix and the selected run files.

Results / Output
Generated files:

- cacm_staticFeatureMatrix.csv
- cacm_outputs/TF-IDF_Results_File.txt
- cacm_outputs/OkapiBM25_Results_File.txt
- cacm_outputs/OkapiTF_Results_File.txt
- cacm_outputs/TF-IDF_Results_File.txt.eval.txt
- cacm_outputs/OkapiBM25_Results_File.txt.eval.txt
- cacm_outputs/OkapiTF_Results_File.txt.eval.txt
- cacm_evaluation_summary.txt

Corpus statistics verified from the CACM parser:

- Documents parsed: 3204
- Queries parsed: 64
- Query IDs with qrels: 52
- Feature matrix rows written: 10721

```
(.venv) PS C:\Users\Hp\Downloads\HW6> Get-Content .\cacm_outputs\TF-IDF_Results_File.txt.eval.t
xt
map all 0.170144
ndcg_cut_10 all 0.268293
P_10 all 0.180769
map 01 0.060044
ndcg_cut_10 01 0.000000
P_10 01 0.000000
map 02 0.004831
ndcg_cut_10 02 0.000000
P_10 02 0.000000
map 03 0.014286
ndcg_cut_10 03 0.000000
P_10 03 0.000000
map 04 0.009762
ndcg_cut_10 04 0.000000
P_10 04 0.000000
```

Evaluation summary (real run values):

- TF-IDF_Results_File.txt: MAP = 0.170144, nDCG@10 = 0.268293, P@10 = 0.180769
- OkapiBM25_Results_File.txt: MAP = 0.286610, nDCG@10 = 0.427835, P@10 = 0.288462
- OkapiTF_Results_File.txt: MAP = 0.159165, nDCG@10 = 0.285230, P@10 = 0.209615

These metrics quantify overall effectiveness (MAP), ranking quality at the top of the list (nDCG@10), and precision at early ranks (P@10).

Challenges Faced

- The CACM collection uses a line-based format rather than XML, so the parsers had to be rewritten to read .I/.T/.W sections.

- The original scoring code scanned every document for every query and model, which was too slow; switching TF-IDF, Okapi TF, and BM25 to postings-based scoring solved this performance problem.
- Feature_Matrix.py still contains legacy AP-era assumptions, but it was patched to avoid the most disruptive path and dependency issues.
- Perl-based trec_eval.pl was not available, so metrics were computed directly in Python to keep the report grounded in actual results.

Conclusion

The project has been successfully switched from the Cranfield dataset to CACM. The CACM pipeline now parses the correct file format, generates run files and a feature matrix, and produces real evaluation values for the report.

References

- CACM dataset in cacm_dataSet/.
- ir_pipeline.py
- build_feature_matrix.py
- train_ranker.py
- Feature_Matrix.py